Duda, R.O., Hart, P.E., and Stork, D.G. 2000, *Pattern Classification*, 2nd ed. (New York: Wiley).

Witten, I.H., and Frank, E. 2005, *Data Mining: Practical Machine Learning Tools and Techniques*, 2nd ed. (San Francisco: Morgan Kaufmann).

Mitchell, T.M. 1997, *Machine Learning* (New York: McGraw-Hill).

Vapnik, V. 1998, *Statistical Learning Theory* (New York: Wiley).

Russell, S., and Norvig, P. 2002, *Artificial Intelligence: A Modern Approach*, 2nd ed. (Upper Saddle River, NJ: Prentice-Hall).

Haykin, S. 1998, *Neural Networks: A Comprehensive Foundation*, 2nd ed. (Upper Saddle River, NJ: Prentice-Hall).

Bishop, C.M. 1996, *Neural Networks for Pattern Recognition* (New York: Oxford University Press).

Korb, K.B., and Nicholson, A.E. 2004, *Bayesian Artificial Intelligence* (Boca Raton, FL: Chapman & Hall/CRC).[1]

Neapolitan, R.E. 1990, *Probabilistic Reasoning in Expert Systems* (New York: Wiley).[2]

Jensen, F.V. 2001, *Bayesian Networks and Decision Graphs* (New York: Springer).[3]

16.1 Gaussian Mixture Models and k-Means Clustering

Gaussian mixture models, so called, are one of the simplest examples of classification by *unsupervised learning*. They are also one of the simplest examples where solution by the *EM (expectation-maximization) algorithm* proves highly successful.

Here is the setup: You are given N data points in an M-dimensional space, usually with M in the range one to a few (say, three or four dimensions, tops). You want to "fit" the data, in this special sense: Find a set of K multivariate Gaussian distributions that best represents the observed distribution of data points. The number K is fixed in advance but the means and covariances of the distributions are unknown,

What makes the exercise "unsupervised" is that you are *not told* which of the N data points come from which of the K Gaussians. Indeed, one of the desired *outputs* is, for each data point n, an estimate of the probability that it came from distribution number k. This probability is denoted $P(k|n)$ or p_{nk}, where (using a zero-based counting scheme) $0 \le k < K$ and $0 \le n < N$. The matrix p_{nk} is sometimes called the *responsibility matrix*, because its entries indicate how much "responsibility" component k has for data point n.

Thus, given the data points, say as an $N \times M$ matrix whose rows are vectors of length M, there are a whole bunch of parameters that we want to estimate:

$$\begin{aligned} \boldsymbol{\mu}_k &\quad \text{(the } K \text{ means, each a vector of length } M\text{)} \\ \boldsymbol{\Sigma}_k &\quad \text{(the } K \text{ covariance matrices, each of size } M \times M\text{)} \\ P(k|n) \equiv p_{nk} &\quad \text{(the } K \text{ probabilities for each of } N \text{ data points)} \end{aligned} \tag{16.1.1}$$

We will also get some additional estimates as by-products: $P(k)$ denotes the fraction of all data points in component k, that is, the probability that a data point chosen at random is in k; $P(\mathbf{x})$ denotes the probability (actually a probability density) of finding a data point at some position $\mathbf{x}$, where $\mathbf{x}$ is the M-dimensional position vector; and $\mathcal{L}$ denotes the overall likelihood of the estimated parameter set.

In fact, $\mathcal{L}$ is the key to the whole problem. $\mathcal{L}$ is defined, as usual, as proportional to the probability of the data set, *given* all the fitted parameters. We find the best values for the parameters by maximizing the likelihood $\mathcal{L}$. You can also think of this as maximizing the posterior probability of the parameters, given uniform or very broad priors.

Let's work backward from $\mathcal{L}$. Since the data points are (assumed) independent, $\mathcal{L}$ is the product of the probabilities of finding a point at each observed position $\mathbf{x}_n$,

$$\mathcal{L} = \prod_n P(\mathbf{x}_n) \tag{16.1.2}$$

We can split $P(\mathbf{x}_n)$ into its contribution from each of the K Gaussians and write

$$P(\mathbf{x}_n) = \sum_k N(\mathbf{x}_n \mid \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k) P(k) \tag{16.1.3}$$

where $N(\mathbf{x} \mid \boldsymbol{\mu}, \boldsymbol{\Sigma})$ is the multivariate Gaussian density,

$$N(\mathbf{x} \mid \boldsymbol{\mu}, \boldsymbol{\Sigma}) = \frac{1}{(2\pi)^{M/2} \det(\boldsymbol{\Sigma})^{1/2}} \exp[-\tfrac{1}{2}(\mathbf{x} - \boldsymbol{\mu}) \cdot \boldsymbol{\Sigma}^{-1} \cdot (\mathbf{x} - \boldsymbol{\mu})] \tag{16.1.4}$$

$P(\mathbf{x}_n)$ is sometimes called the *mixture weight* of the data point $\mathbf{x}_n$. We can "take apart" $P(\mathbf{x}_n)$ into its K individual contributions, giving the individual probabilities

$$p_{nk} \equiv P(k|n) = \frac{N(\mathbf{x}_n \mid \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k) P(k)}{P(\mathbf{x}_n)} \tag{16.1.5}$$

Equations (16.1.2) through (16.1.5) are a prescription for calculating $\mathcal{L}$ and the p_{nk}'s, given the data, and given values for the $\boldsymbol{\mu}_k$'s, $\boldsymbol{\Sigma}_k$'s, and $P(k)$. In the language of the EM algorithm, this is called an *expectation step* or *E-step*.

But how do we get the $\boldsymbol{\mu}_k$'s, $\boldsymbol{\Sigma}_k$'s, and $P(k)$?

Suppose we *knew* the p_{nk}'s. A familiar theorem for the one-dimensional Gaussian distribution is that the maximum likelihood estimate of its mean is just the arithmetic mean of a set of points drawn from it. This theorem straightforwardly generalizes to yield maximum likelihood estimates for the means, and covariance matrices, of multivariate Gaussians. A further small generalization is that, since we know only probabilistically whether a particular point is drawn from a particular Gaussian, we should count only the appropriate fraction p_{nk} of each point. These considerations result in the following maximum likelihood estimates:

$$\begin{aligned} \hat{\boldsymbol{\mu}}_k &= \sum_n p_{nk} \mathbf{x}_n \Big/ \sum_n p_{nk} \\ \hat{\boldsymbol{\Sigma}}_k &= \sum_n p_{nk} (\mathbf{x}_n - \hat{\boldsymbol{\mu}}_k) \otimes (\mathbf{x}_n - \hat{\boldsymbol{\mu}}_k) \Big/ \sum_n p_{nk} \end{aligned} \tag{16.1.6}$$

and, in a similar vein,

$$\hat{P}(k) = \frac{1}{N} \sum_n p_{nk} \tag{16.1.7}$$

"Hats" here denote estimators; however, this is a notational nicety that we will henceforth ignore. Equations (16.1.6) and (16.1.7) are the so-called *maximization step* or *M-step* of the EM algorithm.

What we have motivated thus far is that *right at* the maximum likelihood solution, both the E-step and the M-step relations will hold. That is, the maximum likelihood parameters are a stationary point for both E-steps and M-steps. The power of the EM algorithm derives from the more powerful theorem (beyond our scope to prove here) that, starting from *any* parameter values, an iteration of E-step followed by an M-step will increase the likelihood value $\mathcal{L}$; and that repeated iterations will converge to (at least a local) likelihood maximum. Often, happily, the convergence is to the global maximum.

The EM algorithm, in brief, is thus

- Guess starting values for the $\boldsymbol{\mu}_k$'s, $\boldsymbol{\Sigma}_k$'s, and fractions $P(k)$.
- Repeat: An E-step to get new p_{nk}'s and new $\mathcal{L}$, followed by an M-step to get new $\boldsymbol{\mu}_k$'s, $\boldsymbol{\Sigma}_k$'s, and $P(k)$.
- Quit when the value of $\mathcal{L}$ is no longer changing.

One important practical detail is that the values of the Gaussian density function will often be so small as to underflow to zero. It is therefore important to work with logarithms of these densities, rather than the densities themselves, e.g.,

$$\log N(\mathbf{x} \mid \boldsymbol{\mu}, \boldsymbol{\Sigma}) = -\tfrac{1}{2}(\mathbf{x}-\boldsymbol{\mu})\cdot\boldsymbol{\Sigma}^{-1}\cdot(\mathbf{x}-\boldsymbol{\mu}) - \frac{M}{2}\log(2\pi) - \frac{1}{2}\log\det(\boldsymbol{\Sigma}) \quad (16.1.8)$$

A problem arises with equation (16.1.3), where we need to take the sum of quantities, all of which may be so small as to underflow if ever reconstructed from their logarithms. The solution to this problem is the so-called *log-sum-exp* formula,

$$\log\left(\sum_i \exp(z_i)\right) = z_{\max} + \log\left(\sum_i \exp(z_i - z_{\max})\right) \quad (16.1.9)$$

where the z_i's are the logarithms that we are using to represent small quantities and $z_{\max}$ is their maximum. Equation (16.1.9) guarantees that at least one exponentiation won't underflow, and that any that do could have been neglected anyway.

Figure 16.1.1 shows an example of how the EM algorithm converges to a solution with 1000 two-dimensional data points and four components. As the number of data points increases, the topography of the likelihood space gets smoother, with fewer local minima, so that it becomes more and more likely that the global maximum will be found (as in this case).

You should always inspect an EM solution for reasonableness. If you are getting hung up on an unacceptable local maximum, one strategy is to do a series of independent runs, using K randomly chosen data points as the starting means in each case. (Be sure that you don't duplicate a data point in the starting guesses.) Then pick the best one, i.e., the one that converges to the largest log-likelihood.

Here is a structure that implements the EM algorithm for Gaussian mixture models, given only the data points and initial estimates of the means $\boldsymbol{\mu}_k$. The constructor sets the problem up, and does one initial E-step and M-step. Thereafter, the user alternately calls the `estep()` and `mstep()` routines, until convergence is achieved, as signaled by the return value of `estep()`, the change in log-likelihood, becoming sufficiently small (say, 10^{-6}). The results are then available in the structure members `means`, `resp`, `frac`, and `sig`.

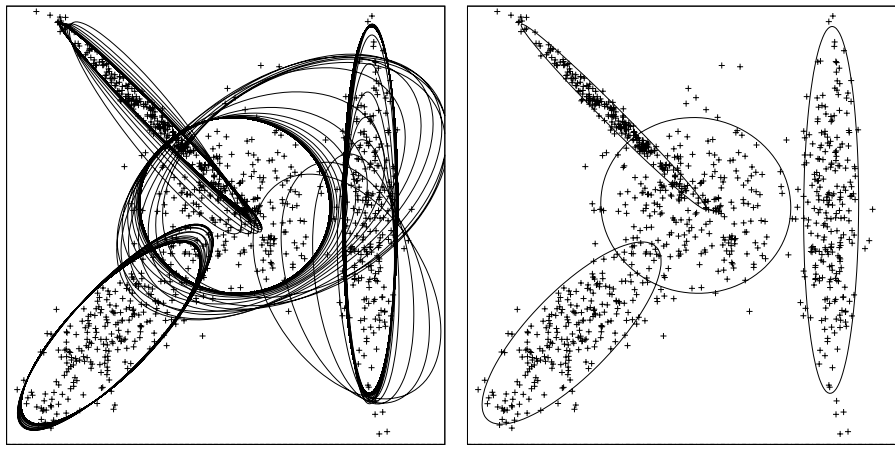

Figure 16.1.1. Example of a Gaussian mixture model in $M = 2$ dimensions, with $N = 1000$ data points and $K = 4$ components. Left: Evolution of the estimated means and covariances, shown as 2-sigma ellipses. The ellipses are plotted after each iteration of an E-step and M-step (see text). Right: The converged result. The leftmost two components converged rapidly. The rightmost component took about 10 iterations to get to near-convergence; only after it had done so did the central component shrink down to a converged result.

gaumixmod.h

```
struct preGaumixmod {
```
For nonwizards, this is basically a typedev of `Mat_mm` as an `mm`×`mm` matrix. For wizards, what is going on is that we need to set a static variable `mmstat` *before* defining `Mat_mm`, and this must happen *before* the `Gaumixmod` constructor is invoked.
```
    static Int mmstat;
    struct Mat_mm : MatDoub {Mat_mm() : MatDoub(mmstat,mmstat) {} };
    preGaumixmod(Int mm) {mmstat = mm;}
};
Int preGaumixmod::mmstat = -1;

struct Gaumixmod : preGaumixmod {
```
Solve for a Gaussian mixture model from a set of data points and initial guesses of k means.
```
    Int nn, kk, mm;                 Nos. of data points, components, and dimensions.
    MatDoub data, means, resp;      Local copies of x_n's, μ_k's, and the p_nk's.
    VecDoub frac, lndets;           P(k)'s and log det Σ_k's.
    vector<Mat_mm> sig;             Σ_k's
    Doub loglike;                   log ℒ.
    Gaumixmod(MatDoub &ddata, MatDoub &mmeans) : preGaumixmod(ddata.ncols()),
    nn(ddata.nrows()), kk(mmeans.nrows()), mm(mmstat), data(ddata), means(mmeans),
    resp(nn,kk), frac(kk), lndets(kk), sig(kk) {
```
Constructor. Arguments are the data points (as rows in a matrix) and initial guesses for the means (also as rows in a matrix).
```
        Int i,j,k;
        for (k=0;k<kk;k++) {
            frac[k] = 1./kk;            Uniform prior on P(k).
            for (i=0;i<mm;i++) {
                for (j=0;j<mm;j++) sig[k][i][j] = 0.;
                sig[k][i][i] = 1.0e-10;     See text at end of this section.
            }
        }
        estep();                        Perform one initial E-step and M-step. User
        mstep();                            is responsible for calling additional steps
    }                                       until convergence is obtained.
    Doub estep() {
```
Perform one E-step of the EM algorithm.
```
        Int k,m,n;
```

```
        Doub tmp,sum,max,oldloglike;
        VecDoub u(mm),v(mm);
        oldloglike = loglike;
        for (k=0;k<kk;k++) {                Outer loop for computing the p_nk's.
            Cholesky choltmp(sig[k]);       Decompose Σ_k in the outer loop.
            lndets[k] = choltmp.logdet();
            for (n=0;n<nn;n++) {            Inner loop for p_nk's.
                for (m=0;m<mm;m++) u[m] = data[n][m]-means[k][m];
                choltmp.elsolve(u,v);       Solve L · v = u.
                for (sum=0.,m=0; m<mm; m++) sum += SQR(v[m]);
                resp[n][k] = -0.5*(sum + lndets[k]) + log(frac[k]);
            }
        }
        At this point we have unnormalized logs of the p_nk's. We need to normalize using
        log-sum-exp and compute the log-likelihood.
        loglike = 0;
        for (n=0;n<nn;n++) {                    Separate normalization for each n.
            max = -99.9e99;                     Log-sum-exp trick begins here.
            for (k=0;k<kk;k++) if (resp[n][k] > max) max = resp[n][k];
            for (sum=0.,k=0; k<kk; k++) sum += exp(resp[n][k]-max);
            tmp = max + log(sum);
            for (k=0;k<kk;k++) resp[n][k] = exp(resp[n][k] - tmp);
            loglike +=tmp;
        }
        return loglike - oldloglike;            When abs of this is small, then we have
    }                                               converged.
    void mstep() {
    Perform one M-step of the EM algorithm.
        Int j,n,k,m;
        Doub wgt,sum;
        for (k=0;k<kk;k++) {
            wgt=0.;
            for (n=0;n<nn;n++) wgt += resp[n][k];
            frac[k] = wgt/nn;                   Equation (16.1.7).
            for (m=0;m<mm;m++) {
                for (sum=0.,n=0; n<nn; n++) sum += resp[n][k]*data[n][m];
                means[k][m] = sum/wgt;          Equation (16.1.6).
                for (j=0;j<mm;j++) {
                    for (sum=0.,n=0; n<nn; n++) {
                        sum += resp[n][k]*
                            (data[n][m]-means[k][m])*(data[n][j]-means[k][j]);
                    }
                    sig[k][m][j] = sum/wgt;     Equation (16.1.6).
                }
            }
        }
    }
};
```

About the only place that `Gaumixmod` can fail algorithmically (as distinct from converging to a poor, local, solution) is by encountering a zero or negative diagonal element in the Cholesky decomposition. As a result, all sins tend to appear, sometimes confusingly, as exceptions at that point in the code. If you are getting such exceptions, here are some possibilities:

- You have duplicated vectors in your initial guesses for the $\boldsymbol{\mu}_k$'s.
- One or more of your $\boldsymbol{\mu}_k$'s is so distant from all data points that it is not "attracting" enough of them to solve for the parameters of its component. Try using random data points as starting guesses, or reduce K.
- You may just have too few data points N to support a nondegenerate model with K components. Reduce K or get more data!

- Rarely, you might want to change the constant `1.0e-10` that initializes the diagonal components of $\mathbf{\Sigma}_k$ in the code. (See discussion under "K-Means Clustering," below.)
- You can reduce the number of parameters in $\mathbf{\Sigma}$, as we now discuss.

Occasionally data are too sparse, or too noisy, to give meaningful results for all the components of the covariance matrices $\mathbf{\Sigma}_k$. In such cases, you can impose simpler covariance models by changing the re-estimation formulas for $\mathbf{\Sigma}$ in equation (16.1.6). One step of simplification is to make $\mathbf{\Sigma}$ diagonal, while still allowing different variances for the different dimensions. The re-estimation formula for the diagonal components of $\mathbf{\Sigma}_k$ is then

$$(\hat{\mathbf{\Sigma}}_k)_{mm} = \sum_n p_{nk}[(\mathbf{x}_n)_m - (\hat{\boldsymbol{\mu}}_k)_m]^2 \Big/ \sum_n p_{nk} \tag{16.1.10}$$

where subscripts m indicate that particular component of the vector. Set nondiagonal components of $\mathbf{\Sigma}_k$ to zero.

Even more drastic, we can replace $\mathbf{\Sigma}_k$ by a single scalar (that is, spherical) variance by using the re-estimation formula

$$(\hat{\mathbf{\Sigma}}_k) = \mathbf{1} \times \left(\sum_n p_{nk} |\mathbf{x}_n - \hat{\boldsymbol{\mu}}_k|^2 \Big/ \sum_n p_{nk} \right) \tag{16.1.11}$$

where $\mathbf{1}$ is the identity matrix.

We have not coded these options in `Gaumixmod`, but they are easy to add.

16.1.1 A Note on the Use of Cholesky Decomposition

It is worth remarking briefly on the use of Cholesky decomposition (§2.9) in this and similar manipulations of multivariate Gaussians.

In the `Gaumixmod` routine above, we need a way of inverting the covariance matrices — or, more precisely, an efficient way to compute expressions like $\mathbf{y} \cdot \mathbf{\Sigma}^{-1} \cdot \mathbf{y}$. Because the covariance matrix $\mathbf{\Sigma}$ is symmetric and positive-definite, the Cholesky decomposition, which has fewer operations than other methods, can be used, giving

$$\mathbf{\Sigma} = \mathbf{L} \cdot \mathbf{L}^T \tag{16.1.12}$$

where $\mathbf{L}$ is a lower triangular matrix, implying

$$Q = \mathbf{y} \cdot \mathbf{\Sigma}^{-1} \cdot \mathbf{y} = \left| \mathbf{L}^{-1} \cdot \mathbf{y} \right|^2 \tag{16.1.13}$$

Since $\mathbf{L}$ is triangular, $\mathbf{L}^{-1} \cdot \mathbf{y}$ can be obtained efficiently by backsubstitution.

Another very convenient use for the decomposition (16.1.12) is in the mundane task of drawing error ellipses, as in Figure 16.1.1 (or, similarly, error ellipsoids in three dimensions). The locus of points $\mathbf{x}$ that are one standard deviation ("1-sigma") away from the mean $\boldsymbol{\mu}$ is given by

$$1 = (\mathbf{x} - \boldsymbol{\mu}) \cdot \mathbf{\Sigma}^{-1} \cdot (\mathbf{x} - \boldsymbol{\mu}) \quad \Rightarrow \quad \left| \mathbf{L}^{-1} \cdot (\mathbf{x} - \boldsymbol{\mu}) \right| = 1 \tag{16.1.14}$$

Now suppose that $\mathbf{z}$ is a point on the unit circle (two dimensions) or unit sphere (three dimensions). Then, by substitution into equation (16.1.14), you can easily see that

$$\mathbf{x} = \mathbf{L} \cdot \mathbf{z} + \boldsymbol{\mu} \tag{16.1.15}$$

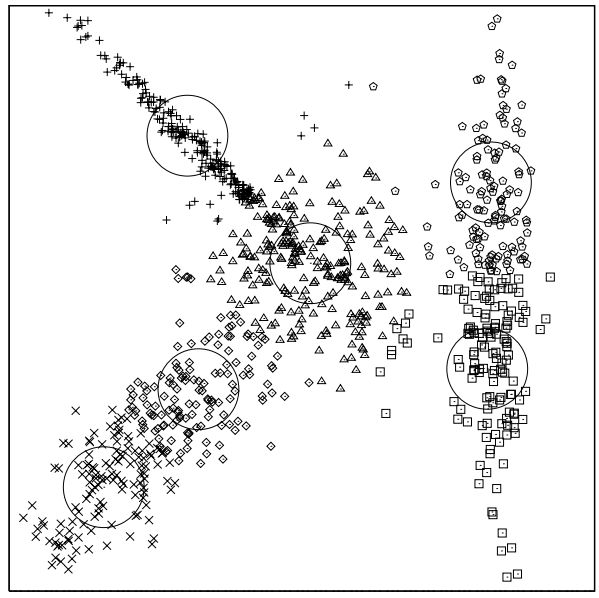

Figure 16.1.2. Output of k-means clustering as applied to the same data as Figure 16.1.1, with $K = 6$ components. The final assignments are shown as different plotted symbols. The centers of the large circles are the locations of the final means. (The radius of those circles is arbitrary, for visibility only.) Unlike Gaussian mixture modeling, k-means clustering can't split a point probabilistically between two components, so many points from the Gaussian at the upper left are mistakenly assigned to the central component. Also, k-means clustering needs more than one component to model a Gaussian with a large aspect ratio, because it clusters by radial distance.

is a point on the 1-sigma locus. Going around the unit circle in $\mathbf{z}$, and using the mapping (16.1.15), gives the desired ellipse. Put a constant 2 in front of the $\mathbf{L}$ in (16.1.15) for 2-sigma ellipses, and so forth.

We already remarked, in §7.4, on the closely related use of Cholesky decomposition to generate multivariate Gaussian deviates from a given covariance matrix.

16.1.2 K-Means Clustering

One interesting simplification of Gaussian mixture modeling has an independent history and is known as *k-means clustering*. We forget about the $\mathbf{\Sigma}_k$ covariances matrices completely, and we forget about probabilistic assignments of data points to components. Instead, each data point gets assigned to one (and only one) of the K components.

The E-step is simply: Assign each data point $\mathbf{x}_n$ to the component k whose mean $\boldsymbol{\mu}_k$ it is closest to, by Euclidean distance.

The M-step is simply: For all k, re-estimate the mean $\boldsymbol{\mu}_k$ as the average of data points $\mathbf{x}_n$ assigned to component k.

The convergence criterion is: Stop when an E-step doesn't change the assignment of any data point (in which case the M-step would also produce unchanged $\boldsymbol{\mu}_k$'s).

Interestingly, convergence is guaranteed — you can't get into an infinite loop of, say, shifting a point back and forth between two components. Despite its simplicity, k-means clustering can be quite useful: It is very fast, and it converges very

rapidly. It can be used as a method to reduce a large number of data points to a much smaller number of "centers," which can then be used as starting points for more sophisticated methods.

For example, you might use k-means clustering to get starting values for a Gaussian mixture model that has difficulty converging to a good, global maximum. If K is the ultimate number of components that you want, you might use k-means to get down to $\sim 3 \times K$ components, then (repeatedly) randomly select K of these as starting guesses for the Gaussian model.

Be alert to the fact that k-means clustering has an intrinsically "spherical" view of the world, because of its Euclidean "nearest-to" assignments. If you have components that might have big aspect ratios, be sure to set K large enough so that these can be represented by several different centers. Figure 16.1.2 shows the same input data as Figure 16.1.1, now clustered by k-means. The Gaussians at the lower left and right have broken up into two centers each. The Gaussian at the upper left is only a single component, because it has had many of its points misclassified into the central component. (A Gaussian mixture model would have assigned those points probabilistically to both components.)

Code for k-means classification is similar to, but much shorter than, the previous code for a Gaussian mixture model:

kmeans.h

```
struct Kmeans {
```
Solve for a k-means clustering model from a set of data points and initial guesses of the means. Output is a set of means and an assignment of each data point to one component.
```
    Int nn, mm, kk, nchg;
    MatDoub data, means;
    VecInt assign, count;
    Kmeans(MatDoub &ddata, MatDoub &mmeans) : nn(ddata.nrows()), mm(ddata.ncols()),
    kk(mmeans.nrows()), data(ddata), means(mmeans), assign(nn), count(kk) {
```
Constructor. Arguments are the data points (as rows in a matrix), and initial guesses for the means (also as rows in a matrix).
```
        estep();
        mstep();
    }
```
Perform one initial E-step and M-step. User is responsible for calling additional steps until convergence is obtained.
```
    Int estep() {
```
Perform one E-step.
```
        Int k,m,n,kmin;
        Doub dmin,d;
        nchg = 0;
        for (k=0;k<kk;k++) count[k] = 0;
        for (n=0;n<nn;n++) {
            dmin = 9.99e99;
            for (k=0;k<kk;k++) {
                for (d=0.,m=0; m<mm; m++) d += SQR(data[n][m]-means[k][m]);
                if (d < dmin) {dmin = d; kmin = k;}
            }
            if (kmin != assign[n]) nchg++;
            assign[n] = kmin;
            count[kmin]++;
        }
        return nchg;
    }
    void mstep() {
```
Perform one M-step.
```
        Int n,k,m;
        for (k=0;k<kk;k++) for (m=0;m<mm;m++) means[k][m] = 0.;
        for (n=0;n<nn;n++) for (m=0;m<mm;m++) means[assign[n]][m] += data[n][m];
        for (k=0;k<kk;k++) {
```